

Walrus Sessions 8 — Estratégia para ganhar

Hackathon: "Chatbots That Remember" · 18/set → 9/out/2026 · DeepSurge · prêmios em WAL. Projeto separado do [hackatons/hackmemo](#) (que não é alterado).

1. O que o hackathon premia (fonte: páginas públicas; rubrica oficial NÃO verificada)

Trilha	Prêmio (WAL)	O que vale
Best Chatbot	500 / 250 / 150	produção real + recall verificável
Beyond the Big Two	2 × 150	LLM principal que não é OpenAI nem Claude (Gemini, DeepSeek, Qwen...)
Best Article	3 × 100	write-up técnico + antes/depois
Bug Bounty	5 × 100	bug reproduzível (ex.: corrida SEAL)

As trilhas acumulam. Um projeto só pode disputar as quatro.

Padrão dos concorrentes já publicados (walcoach, dosedaughter, walrus-social-agent, walrus-memory-chatbot, hippo, walbot): mainnet, ≥10 blobs, ≥3 usuários/personas, link de walruscan por memória, script de eval com/sem recall, artigo publicado. Isso é o **piso**, não o diferencial.

2. Onde o campo está saturado e onde está vazio

Saturado: coach pessoal, suporte ao cliente, assistente de preferências, bot social. Todos são "um usuário, um bot, lembra do que eu gosto".

Vazio (ninguém explorou):

- Memória de **grupo** (várias pessoas, uma verdade compartilhada).
- **Mudança de fato** em armazenamento append-only (sem update/delete): como o "último combinado vale". O SDK tem `sort`: `"recent"` e `scoringWeights` (recência, meia-vida, importância) que quase ninguém usa.
- Imutabilidade como **recurso** (prestação de contas), não como limitação.

3. Ideia recomendada: "Palavra" — livro-razão de compromissos em grupos

Bot em grupos de WhatsApp/Telegram que registra combinados ("Ana fica de mandar o orçamento até sexta"), responde "quem ficou de quê?" e "o que decidimos sobre X?", e cobra pendências. Cada compromisso vira um blob no Walrus com link de prova.

Por que ganha:

- Memória faz trabalho real: sem ela o bot não sabe o que foi combinado semana passada.

- Imutabilidade é o produto: ninguém reescreve o que foi combinado.
- 3 usuários vêm de graça (um grupo de 3+ pessoas já cumpre).
- Mudanças ("adiamos para sábado") mostram o caso técnico difícil resolvido.

Decisão técnica-chave: memória isolada é por **owner + namespace**, então **um namespace por grupo (grp:<id>)** sob **uma única conta do bot** basta. Não depende de compartilhar conta entre pessoas (risco que eu tinha levantado). A autoria vai no texto do fato ("Ana se comprometeu a...").

Diferenciais técnicos (o que separa de um chatbot com preferências)

1. **Supersessão em log imutável:** cada fato tem tipo (**compromisso**, **alteração**, **conclusão**) e referência ao blob que substitui. Recall com **sort: "recent" + scoringWeights.recency**, e o bot resolve "estado atual" no código, não no LLM.
2. **Recibos verificáveis:** toda resposta cita o(s) blob(s) usados, com link de walruscan. Judges clicam e veem o blob criptografado.
3. **Guarda determinística** (à la DoseDaughter): lembrete de prazo vencido dispara por código a partir da memória, antes do LLM.
4. **Eval reprodutível:** **npm run eval** simula grupos em 2 sessões, com recall desligado vs ligado, e imprime acertos. Usar **MemWalMock** em CI e mainnet no relatório.
5. **Injeção segura:** memória entra no prompt num bloco delimitado e tratado como dado não confiável (o SDK tem **untrusted-memory** em **/ai**).

Estratégia por trilha

- **Best Chatbot:** deploy real em grupos reais por vários dias, ledger de blobs, transcrições.
- **Beyond the Big Two:** Gemini (ou DeepSeek/Qwen) como LLM único. Nenhum Claude/GPT no caminho.
- **Best Article:** "Memória append-only: como lidar com fatos que mudam" com o antes/depois medido pelo eval. Publicar em DEV/Medium (concorrentes usaram Inkay e Medium).
- **Bug Bounty:** registrar cada anomalia com reprodução mínima: recall vazio logo após escrita, empate de timestamp, timeouts. Já há relatos públicos (issue #943 do SDK Python), então buscar bugs novos e reproduzíveis, não repetir os conhecidos.

4. Alternativas (se Palavra não empolgar)

- **Ata viva:** reuniões/assembleias (condomínio, associação) com decisões imutáveis.
- **Cliente lembra:** WhatsApp de pequeno comércio que lembra pedidos e preferências de cada cliente. Distribuição enorme no Brasil, mas cai no padrão "preferências".
- **Memória portátil entre apps:** mesma conta em WhatsApp + web + CLI. Boa demo técnica, dor menos concreta.

5. Riscos e pontos a verificar ANTES de codar

- ☐ Rubrica oficial de julgamento na página do evento no DeepSurge (não consegui abrir).
- ☐ Mainnet é obrigatório? Custo em SUI/WAL para conta e escrita; há conta patrocinada?
- ☐ Limites de tamanho de texto e rate limit do relayer (não documentados na referência).
- ☐ Baileys viola os termos do WhatsApp e pode banir o número. Plano B: Telegram + web.
- ☐ Privacidade: gravar só mensagens endereçadas ao bot (**/combinado**, menção), com aviso no grupo.

- ☐ Latência: escrita assíncrona chega a ~26s e o recall pode voltar vazio logo após escrever. Precisa de outbox local + retry com backoff.
- ☐ "Gemini 1.5 Flash" aparece no resumo da trilha Beyond the Big Two. Confirmar se o modelo usado (ex.: 2.x) qualifica.

6. Cronograma (16 dias, hoje 23/set → 9/out)

- D1–2: conta MemWal em mainnet, esqueleto, modelo de fato + supersessão, outbox/retry.
- D3–5: canal (Telegram primeiro, WhatsApp depois), comandos, recibos com walruscan.
- D6–7: eval com/sem recall + guarda determinística de prazos.
- D8–12: uso real em grupos (mínimo 3 pessoas), coletar evidência e bugs.
- D13–14: artigo + README com ledger de blobs e transcrições.
- D15–16: buffer, deploy estável, submissão antes de 9/out.